LAPORAN PRAKTIKUM

PRAKTIKUM PENAMBANGAN DATA

PERTEMUAN 4

REGRESI



Disusun oleh:

Nama	:	Hanan Fijananto
NIM	:	24/538946/SV/24555
Kelas	:	A2
Dosen Pengampu	:	Dr.Imam Fahrurrozi, S.T., M.Cs.

PROGRAM STUDI D-IV TEKNOLOGI REKAYASA PERANGKAT LUNAK

DEPARTEMEN TEKNIK ELEKTRO DAN INFORMATIKA

SEKOLAH VOKASI

UNIVERSITAS GADJAH MADA

YOGYAKARTA

2026

A Tujuan

1. Mahasiswa mampu memahami konsep dasar regresi dan algoritma regresi.
2. Mahasiswa mengetahui prinsip kerja berbagai algoritma regresi, khususnya Linear Regression, Decision Tree Regression, dan Random Forest Regression
3. Mahasiswa mampu melakukan evaluasi model regresi menggunakan metrik RMSE dan R^2.
4. Mahasiswa dapat menerapkan berbagai algoritma regresi untuk membangun model.

B Dasar Teori

B.1 Regresi

Regresi merupakan salah satu metode dalam pembelajaran mesin dan statistika yang digunakan untuk memodelkan hubungan antara variabel independen atau variabel prediktor dengan variabel dependen atau variabel target yang memiliki nilai kontinu. Tujuan utama regresi adalah menghasilkan suatu model yang dapat digunakan untuk memperkirakan nilai variabel target berdasarkan informasi yang terdapat pada variabel prediktor. Dalam konteks pembelajaran mesin, model regresi dibangun menggunakan data pelatihan untuk mempelajari pola hubungan antara fitur masukan dan nilai target, kemudian digunakan untuk melakukan prediksi terhadap data baru.

Secara umum, permasalahan regresi dapat dinyatakan sebagai pasangan data (x_i, y_i), dengan x_i merupakan vektor fitur pada pengamatan ke-i dan y_i merupakan nilai target yang ingin diprediksi. Model regresi berusaha menemukan suatu fungsi $\hat{f}$ sehingga nilai prediksi $\hat{y}_i = \hat{f}(x_i)$ memiliki kedekatan dengan nilai aktual y_i. Salah satu tujuan utama proses pembelajaran tersebut adalah meminimalkan kesalahan

antara nilai aktual dan nilai hasil prediksi [1].

Regresi berbeda dengan klasifikasi. Pada klasifikasi, variabel target berupa kategori atau kelas, sedangkan pada regresi variabel target umumnya berupa nilai numerik kontinu. Oleh karena itu, regresi dapat digunakan pada permasalahan seperti prediksi harga, jumlah permintaan, pendapatan, suhu, konsumsi energi, dan berbagai variabel numerik lainnya.

B.2 Algoritma Regresi

Algoritma regresi merupakan metode atau pendekatan yang digunakan untuk membentuk fungsi prediksi berdasarkan data yang tersedia. Algoritma tersebut mempelajari pola hubungan antara variabel prediktor dan variabel target melalui proses pelatihan. Setelah model terbentuk, model dapat digunakan untuk menghasilkan prediksi terhadap data yang belum pernah digunakan dalam proses pelatihan.

Terdapat berbagai algoritma yang dapat digunakan untuk menyelesaikan permasalahan regresi. Pemilihan algoritma bergantung pada karakteristik data, hubungan antara variabel, kompleksitas pola, serta kebutuhan interpretasi model. Beberapa algoritma yang umum digunakan antara lain Linear Regression, Decision Tree Regression, Random Forest Regression, Support Vector Regression, dan berbagai algoritma berbasis ensemble lainnya [1].

Dalam penelitian ini, algoritma regresi yang digunakan adalah Linear Regression, Decision Tree Regression, dan Random Forest Regression. Ketiga algoritma tersebut memiliki pendekatan yang berbeda dalam memodelkan hubungan antara variabel prediktor dan variabel target.

B.3 Linear Regression

Linear Regression atau regresi linear merupakan salah satu metode regresi yang digunakan untuk memodelkan hubungan linear antara variabel prediktor dengan

variabel target. Metode ini mengasumsikan bahwa perubahan pada variabel target dapat dijelaskan melalui kombinasi linear dari variabel prediktor.

Pada regresi linear dengan satu variabel prediktor, bentuk umum model dapat dituliskan sebagai:

$$\hat{y} = \beta_0 + \beta_1 x \tag{1}$$

dengan $\hat{y}$ merupakan nilai prediksi, β_0 merupakan intercept atau konstanta, β_1 merupakan koefisien regresi, dan x merupakan variabel prediktor. Pada regresi linear berganda yang menggunakan lebih dari satu variabel prediktor, persamaan dapat dituliskan sebagai:

$$\hat{y} = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \cdots + \beta_p x_p \tag{2}$$

Koefisien regresi diperoleh melalui proses estimasi yang bertujuan menemukan parameter model sehingga nilai prediksi memiliki kesalahan sekecil mungkin terhadap nilai aktual. Salah satu pendekatan yang umum digunakan adalah Ordinary Least Squares (OLS), yaitu dengan meminimalkan jumlah kuadrat residual antara nilai aktual dan nilai prediksi [1].

$$\underset{\beta}{\text{minimize}} \sum_{i=1}^{n} (y_i - \hat{y}_i)^2 \tag{3}$$

Residual merupakan selisih antara nilai aktual dengan nilai prediksi yang dapat dirumuskan sebagai:

$$e_i = y_i - \hat{y}_i \tag{4}$$

Implementasi Linear Regression pada *Scikit-learn* menyediakan estimator `LinearRegression` yang digunakan untuk melakukan prediksi berdasarkan model linear. Evaluasi model regresi tersebut juga dapat dilakukan menggunakan koefisien determinasi R^2 [2].

B.4 Decision Tree Regression

Decision Tree Regression merupakan metode regresi berbasis pohon keputusan yang membagi ruang data menjadi beberapa bagian berdasarkan aturan pemisahan tertentu. Berbeda dengan Linear Regression yang membentuk hubungan linear, Decision Tree Regression dapat memodelkan hubungan yang tidak linear antara variabel prediktor dan variabel target.

Proses pembentukan Decision Tree Regression dilakukan dengan melakukan pemisahan data secara bertahap berdasarkan fitur dan nilai ambang tertentu. Setiap pemisahan menghasilkan beberapa subset data yang kemudian dapat kembali dipisahkan hingga memenuhi kondisi penghentian tertentu. Pada bagian akhir pohon terdapat *leaf* atau simpul terminal yang digunakan untuk menghasilkan nilai prediksi.

Pada permasalahan regresi, salah satu kriteria yang dapat digunakan untuk menentukan kualitas pemisahan adalah Mean Squared Error (MSE). Untuk suatu node m yang memiliki n_m observasi, MSE dapat dinyatakan sebagai:

$$MSE_m = \frac{1}{n_m} \sum_{y_i \in Q_m} (y_i - \bar{y}_m)^2 \tag{5}$$

dengan Q_m merupakan kumpulan data yang berada pada node m dan $\bar{y}_m$ merupakan nilai rata-rata target pada node tersebut. Pemisahan dipilih berdasarkan pengurangan kesalahan yang dihasilkan oleh pembagian data. Pada simpul terminal, nilai prediksi umumnya berupa rata-rata nilai target dari data yang berada pada simpul tersebut ketika menggunakan kriteria squared error [3].

Decision Tree Regression memiliki kemampuan untuk menangkap hubungan nonlinear dan interaksi antarvariabel tanpa memerlukan transformasi matematis tertentu. Namun, pohon keputusan yang terlalu kompleks dapat mengalami *overfitting*, yaitu kondisi ketika model sangat baik dalam memprediksi data pelatihan tetapi memiliki kemampuan generalisasi yang rendah terhadap data baru.

B.5 Random Forest Regression

Random Forest Regression merupakan algoritma ensemble yang menggabungkan sejumlah Decision Tree untuk menghasilkan prediksi yang lebih stabil dan umumnya memiliki kemampuan generalisasi yang lebih baik dibandingkan sebuah pohon keputusan tunggal. Konsep Random Forest diperkenalkan oleh Breiman sebagai kumpulan prediktor berbasis pohon yang masing-masing dibangun menggunakan unsur pengacakan tertentu [4].

Dalam Random Forest Regression, beberapa pohon keputusan dibangun menggunakan variasi data pelatihan dan pemilihan fitur secara acak. Setiap pohon menghasilkan prediksi terhadap suatu observasi, kemudian hasil prediksi dari seluruh pohon digabungkan. Untuk permasalahan regresi, penggabungan tersebut umumnya dilakukan dengan menghitung rata-rata prediksi dari seluruh pohon.

Jika terdapat B pohon keputusan dan prediksi dari pohon ke-b adalah $\hat{f}_b(x)$, maka prediksi Random Forest dapat dinyatakan sebagai:

$$\hat{f}_{RF}(x)\frac{1}{B}\sum_{b=1}^{B}\hat{f}_b(x) \tag{6}$$

Penggunaan banyak pohon dengan variasi data dan fitur dapat mengurangi ketergantungan model terhadap satu struktur pohon tertentu. Random Forest juga dapat digunakan untuk memperkirakan pentingnya variabel dalam proses prediksi. Menurut Breiman, pendekatan Random Forest dapat digunakan baik untuk permasalahan klasifikasi maupun regresi [4].

Meskipun demikian, Random Forest memiliki kompleksitas yang lebih tinggi dibandingkan Linear Regression. Model juga cenderung lebih sulit diinterpretasikan karena hasil prediksi berasal dari kombinasi banyak pohon keputusan. Oleh karena itu, pemilihan Random Forest perlu mempertimbangkan kebutuhan antara kemampuan prediksi, kompleksitas model, dan interpretabilitas.

B.6 Metrik Evaluasi Regresi

Metrik evaluasi regresi digunakan untuk mengukur seberapa baik model menghasilkan prediksi terhadap nilai aktual. Evaluasi diperlukan karena nilai prediksi yang dihasilkan model belum dapat menunjukkan kualitas model tanpa dibandingkan dengan nilai sebenarnya.

Beberapa metrik yang umum digunakan dalam evaluasi regresi adalah Mean Absolute Error (MAE), Mean Squared Error (MSE), Root Mean Squared Error (RMSE), Mean Absolute Percentage Error (MAPE), dan Coefficient of Determination (R^2). Scikit-learn menyediakan berbagai metrik tersebut sebagai bagian dari evaluasi model regresi [5].

Dalam penelitian ini, metrik yang digunakan adalah RMSE dan R^2. Penggunaan kedua metrik tersebut memungkinkan evaluasi dilakukan dari dua sudut pandang, yaitu besarnya kesalahan prediksi dan kemampuan model dalam menjelaskan variasi nilai target.

B.7 Root Mean Squared Error (RMSE)

Root Mean Squared Error (RMSE) merupakan metrik evaluasi yang digunakan untuk mengukur besarnya rata-rata kesalahan prediksi dengan memberikan penalti yang lebih besar terhadap kesalahan yang bernilai besar. RMSE diperoleh dengan menghitung akar kuadrat dari rata-rata kuadrat selisih antara nilai aktual dan nilai prediksi.

RMSE dapat dirumuskan sebagai:

$$RMSE = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2} \tag{7}$$

dengan n merupakan jumlah observasi, y_i merupakan nilai aktual pada observasi ke-i, dan $\hat{y}_i$ merupakan nilai prediksi model.

Nilai RMSE yang semakin kecil menunjukkan bahwa nilai prediksi model se-

makin dekat dengan nilai aktual. Nilai RMSE sebesar 0 menunjukkan bahwa seluruh prediksi sama dengan nilai aktual. Karena menggunakan operasi kuadrat, RMSE memberikan pengaruh yang lebih besar terhadap kesalahan prediksi yang besar dibandingkan kesalahan yang kecil. NIST juga menjelaskan akar dari rata-rata kuadrat residual sebagai salah satu ukuran untuk mengevaluasi kualitas kecocokan suatu model, dengan nilai yang lebih kecil menunjukkan kecocokan yang lebih baik [6].

Salah satu kelebihan RMSE adalah satuannya sama dengan variabel target sehingga hasilnya relatif mudah diinterpretasikan. Sebagai contoh, apabila variabel target memiliki satuan rupiah, maka RMSE juga memiliki satuan rupiah.

B.8 Coefficient of Determination (R^2)

Coefficient of Determination atau koefisien determinasi (R^2) merupakan metrik yang digunakan untuk mengukur proporsi variasi pada variabel target yang dapat dijelaskan oleh model berdasarkan variabel prediktor. Dengan demikian, R^2 memberikan informasi mengenai seberapa baik model menjelaskan variasi nilai target dibandingkan dengan model sederhana yang hanya menggunakan nilai rata-rata target.

Nilai R^2 dapat dirumuskan sebagai:

$$R^2 = 1 - \frac{\sum_{i=1}^{n}(y_i - \hat{y} * i)^2}{\sum *i = 1^n (y_i - \bar{y})^2} \tag{8}$$

dengan y_i merupakan nilai aktual, $\hat{y}_i$ merupakan nilai prediksi, dan $\bar{y}$ merupakan rata-rata nilai aktual.

Nilai R^2 maksimum adalah 1. Nilai yang semakin mendekati 1 menunjukkan bahwa model mampu menjelaskan variasi data dengan lebih baik. Nilai $R^2 = 0$ menunjukkan bahwa model memiliki performa yang setara dengan model sederhana yang selalu memprediksi nilai rata-rata target. Sementara itu, nilai R^2 dapat bernilai

negatif apabila performa model lebih buruk dibandingkan prediksi menggunakan nilai rata-rata target [5].

Nilai R^2 tidak secara langsung menunjukkan besarnya kesalahan prediksi dalam satuan asli variabel target. Oleh karena itu, penggunaan R^2 sebaiknya dipadukan dengan metrik berbasis error seperti RMSE. RMSE memberikan informasi mengenai besar kesalahan prediksi, sedangkan R^2 memberikan informasi mengenai kemampuan model dalam menjelaskan variasi data.

B.9 Perbandingan Algoritma Regresi

Ketiga algoritma yang digunakan memiliki karakteristik yang berbeda. Linear Regression sesuai digunakan ketika hubungan antara variabel prediktor dan target cenderung linear serta interpretasi koefisien model menjadi salah satu kebutuhan utama. Decision Tree Regression lebih fleksibel dalam menangkap hubungan non-linear, tetapi pohon yang terlalu dalam berpotensi mengalami *overfitting*. Sementara itu, Random Forest Regression menggabungkan banyak Decision Tree sehingga dapat menghasilkan model yang lebih robust terhadap variasi data, tetapi memiliki kompleksitas dan kebutuhan komputasi yang lebih tinggi.

Oleh karena itu, pemilihan model tidak hanya didasarkan pada algoritma yang digunakan, tetapi juga pada hasil evaluasi menggunakan data pengujian. Model dengan nilai RMSE yang lebih rendah dan nilai R^2 yang lebih tinggi secara umum menunjukkan performa prediksi yang lebih baik, dengan tetap mempertimbangkan karakteristik data dan tujuan penelitian.

C Hasil dan Pembahasan

C.1 Import Library

Sebelum mengimport library, pastikan library yang dibutuhkan sudah terinstall, salah satunya yaitu library seaborn. Praktikan dapat menginstall library seaborn

dengan menggunakan perintah berikut:

```
pip install seaborn
```

```
import pandas as pd
import matplotlib.pyplot as plt
import numpy as np
import seaborn as sns

from sklearn. linear_model import LinearRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error
from sklearn.metrics import r2_score
from sklearn. tree import DecisionTreeRegressor
from sklearn.ensemble import RandomForestRegressor
```

Gambar 1: Perintah Install Library Seaborn

C.2 Load Dataset

Pada praktikum ini, dataset yang digunakan adalah dataset harga rumah dengan link kaggle berikut `https://www.kaggle.com/datasets/shivachandel/kc-house-data`, atau praktikan juga dapat mengakses dataset secara langsung melalui repository yang telah disediakan.

```
df=pd.read_csv('https://raw.githubusercontent.com/ganjar87
/data_science_practice/main/kc_house_data.csv')
```

C.3 Exploratory Data Analysis (EDA)

C.3.1 Melihat 5 baris pertama dan terakhir

```
df.head()
```
✓ 0.0s Python

	id	date	price	bedrooms	bathrooms	sqft_living	sqft_lot	floors	waterfront	v
0	7129300520	20141013T000000	221900.0	3	1.00	1180	5650	1.0	0	
1	6414100192	20141209T000000	538000.0	3	2.25	2570	7242	2.0	0	
2	5631500400	20150225T000000	180000.0	2	1.00	770	10000	1.0	0	
3	2487200875	20141209T000000	604000.0	4	3.00	1960	5000	1.0	0	
4	1954400510	20150218T000000	510000.0	3	2.00	1680	8080	1.0	0	

5 rows × 21 columns

```
df.tail()
```
✓ 0.0s Python

	id	date	price	bedrooms	bathrooms	sqft_living	sqft_lot	floors	waterfron
21608	263000018	20140521T000000	360000.0	3	2.50	1530	1131	3.0	
21609	6600060120	20150223T000000	400000.0	4	2.50	2310	5813	2.0	
21610	1523300141	20140623T000000	402101.0	2	0.75	1020	1350	2.0	
21611	291310100	20150116T000000	400000.0	3	2.50	1600	2388	2.0	
21612	1523300157	20141015T000000	325000.0	2	0.75	1020	1076	2.0	

5 rows × 21 columns

Gambar 2: Melihat 5 baris pertama dan terakhir

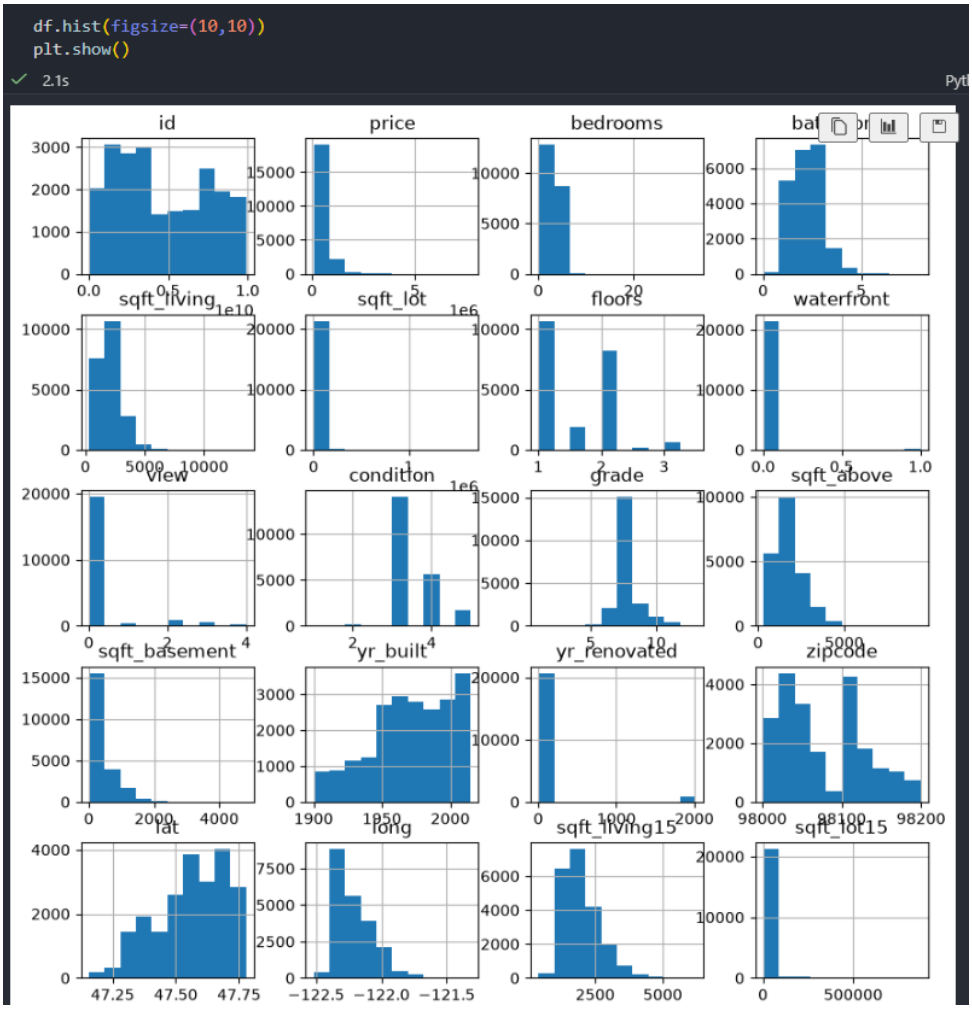
C.3.2 Melihat deskripsi statistik dengan df.describe()

```
df.describe()
```
✓ 0.1s Python

	id	price	bedrooms	bathrooms	sqft_living	sqft_lot	floors	21
count	2.161300e+04	2.161300e+04	21613.000000	21613.000000	21613.000000	2.161300e+04	21613.000000	
mean	4.580302e+09	5.400881e+05	3.370842	2.114757	2079.899736	1.510697e+04	1.494309	
std	2.876566e+09	3.671272e+05	0.930062	0.770163	918.440897	4.142051e+04	0.539989	
min	1.000102e+06	7.500000e+04	0.000000	0.000000	290.000000	5.200000e+02	1.000000	
25%	2.123049e+09	3.219500e+05	3.000000	1.750000	1427.000000	5.040000e+03	1.000000	
50%	3.904930e+09	4.500000e+05	3.000000	2.250000	1910.000000	7.618000e+03	1.500000	
75%	7.308900e+09	6.450000e+05	4.000000	2.500000	2550.000000	1.068800e+04	2.000000	
max	9.900000e+09	7.700000e+06	33.000000	8.000000	13540.000000	1.651359e+06	3.500000	

Gambar 3: Deskripsi Statistik

C.3.3 Melihat histogram



Gambar 4: Histogram

C.3.4 Mengecek korelasi atribut numeric dengan df.corr(numeric_only = True)

```
df.corr(numeric_only=True)
```

✓ 0.0s Python

	id	price	bedrooms	bathrooms	sqft_living	sqft_lot	floors	waterfront	
id	1.000000	-0.016762	0.001286	0.005160	-0.012258	-0.132109	0.018525	-0.002721	0.0
price	-0.016762	1.000000	0.308350	0.525138	0.702035	0.089661	0.256794	0.266369	0.3
bedrooms	0.001286	0.308350	1.000000	0.515884	0.576671	0.031703	0.175429	-0.006582	0.0
bathrooms	0.005160	0.525138	0.515884	1.000000	0.754665	0.087740	0.500653	0.063744	0.1
sqft_living	-0.012258	0.702035	0.576671	0.754665	1.000000	0.172826	0.353949	0.103818	0.2
sqft_lot	-0.132109	0.089661	0.031703	0.087740	0.172826	1.000000	-0.005201	0.021604	0.0
floors	0.018525	0.256794	0.175429	0.500653	0.353949	-0.005201	1.000000	0.023698	0.0
waterfront	-0.002721	0.266369	-0.006582	0.063744	0.103818	0.021604	0.023698	1.000000	0.4
view	0.011592	0.397293	0.079532	0.187737	0.284611	0.074710	0.029444	0.401857	1.0
condition	-0.023783	0.036362	0.028472	-0.124982	-0.058753	-0.008958	-0.263768	0.016653	0.0
grade	0.008130	0.667434	0.356967	0.664983	0.762704	0.113621	0.458183	0.082775	0.2
sqft_above	-0.010842	0.605567	0.477600	0.685342	0.876597	0.183512	0.523885	0.072075	0.1
sqft_basement	-0.005151	0.323816	0.303093	0.283770	0.435043	0.015286	-0.245705	0.080588	0.2
yr_built	0.021380	0.054012	0.154178	0.506019	0.318049	0.053080	0.489319	-0.026161	-0.0
yr_renovated	-0.016907	0.126434	0.018841	0.050739	0.055363	0.007644	0.006338	0.092885	0.1
zipcode	-0.008224	-0.053203	-0.152668	-0.203866	-0.199430	-0.129574	-0.059121	0.030285	0.0
lat	-0.001891	0.307003	-0.008931	0.024573	0.052529	-0.085683	0.049614	-0.014274	0.0
long	0.020799	0.021626	0.129473	0.223042	0.240223	0.229521	0.125419	-0.041910	-0.0
sqft_living15	-0.002901	0.585379	0.391638	0.568634	0.756420	0.144608	0.279885	0.086463	0.2
sqft_lot15	-0.138798	0.082447	0.029244	0.087175	0.183286	0.718557	-0.011269	0.030703	0.0

Gambar 5: Korelasi Atribut Numerik

C.3.5 Mengecek missing value dengan df.isnull().sum()

```
df.isnull().sum()
✓ 0.0s
id              0
date            0
price           0
bedrooms        0
bathrooms       0
sqft_living     0
sqft_lot        0
floors          0
waterfront      0
view            0
condition       0
grade           0
sqft_above      0
sqft_basement   0
yr_built        0
yr_renovated    0
zipcode         0
lat             0
long            0
sqft_living15   0
sqft_lot15      0
dtype: int64
```

Gambar 6: Missing Value

Dapat dilihat pada gambar 6 bahwa tidak terdapat missing value pada dataset ini. Sehingga tidak perlu dilakukan penanganan missing value.

C.3.6 Mengecek atribut kategorikal

```
df_X = df.drop(['id', 'date','price'], axis=1)
df_y = df['price']
cats = df_X.select_dtypes(include=['object', 'bool']).columns
print(cats)

✓ 0.0s

Index([], dtype='str')
```

Gambar 7: Atribut Kategorikal

C.4 Modelling

C.4.1 Linear Regression

```
df_X = df.drop(['id', 'date','price'], axis=1)
df_y = df['price']

X = df_X.astype(float).values
y =df_y.astype(float).values

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)
reg = LinearRegression()
reg.fit(X_train, y_train)

print('coef of determination training ', reg.score(X_train, y_train))
print('coef of determination testing ', reg.score(X_test, y_test))

print('coefficient')
print(reg.coef_)
print('intercept')
print(reg.intercept_)

print('prediction')
y_pred = reg.predict(X_test)
print(y_pred[:10])
print('real value')
print(y_test[0:10])
```

Gambar 8: Linear Regression

Kode tersebut diawali dengan memisahkan variabel prediktor dan variabel target, yaitu menghapus kolom `id`, `date`, dan `price` dari fitur menggunakan `drop()`,

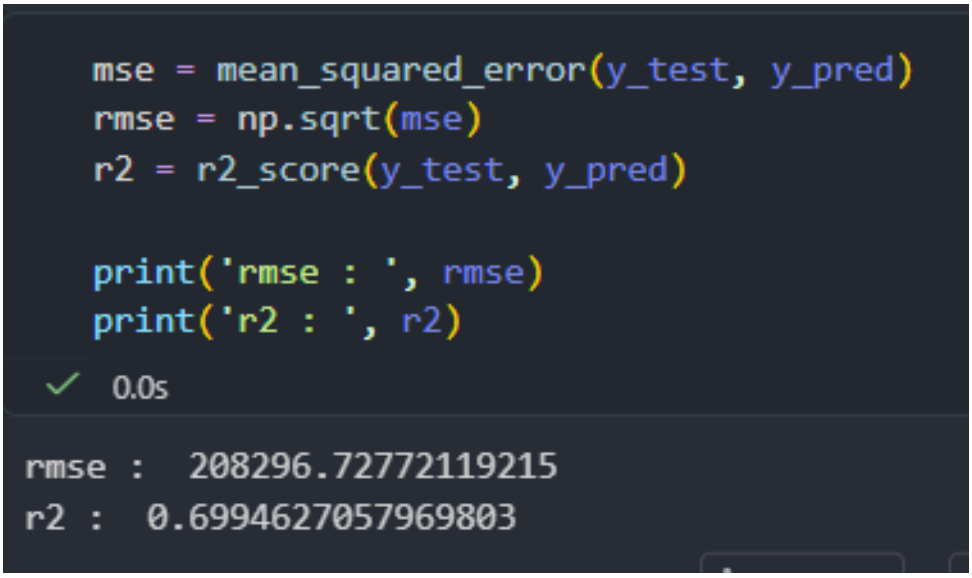
sedangkan kolom `price` digunakan sebagai target. Data kemudian dikonversi menjadi tipe `float` dan *NumPy array* agar dapat diproses oleh model. Selanjutnya, dataset dibagi menjadi data latih sebesar 70% dan data uji sebesar 30% menggunakan `train_test_split()` dengan `random_state=42` agar pembagian data konsisten. Model *Linear Regression* dibuat menggunakan `LinearRegression()` dan dilatih dengan data latih menggunakan `fit()`. Setelah pelatihan, metode `score()` digunakan untuk memperoleh nilai *coefficient of determination* (R^2) pada data latih dan data uji. Atribut `coef_` digunakan untuk memperoleh koefisien regresi, sedangkan `intercept_` digunakan untuk memperoleh nilai konstanta model. Terakhir, metode `predict()` digunakan untuk menghasilkan prediksi harga berdasarkan data uji, kemudian hasil prediksi tersebut dibandingkan dengan nilai aktual yang tersimpan pada `y_test`.

Outputnya dapat dilihat pada gambar 9.

```
coef of determination training  0.6995155846436756
coef of determination testing  0.6994627057969803
coefficient
[-3.43081477e+04  4.03129700e+04  1.12001375e+02  9.91841247e-02
  5.27154218e+03  5.43877177e+05  5.50830616e+04  2.31460673e+04
  9.49081794e+04  7.22190668e+01  3.97823082e+01 -2.59441847e+03
  2.19209734e+01 -5.56358731e+02  5.95216324e+05 -1.96904658e+05
  1.62077488e+01 -3.30430480e-01]
intercept
6641646.708119173
prediction
[ 458597.06764164  748993.75994819 1243303.75799043 1665116.9509542
  737302.0574173   283239.58524984  831732.87582313  495383.02095339
  385779.81919027  474179.42285136]
real value
[ 365000.  865000. 1038000. 1490000.  711000.  211000.  790000.  680000.
  384500.  605000.]
```

Gambar 9: Output Linear Regression

Tahap selanjutnya yaitu evaluasi model menggunakan RMSE dan R^2.



Gambar 10: Evaluasi Model Linear Regression

Kode tersebut digunakan untuk menghitung metrik evaluasi model regresi, yaitu *Root Mean Squared Error* (RMSE) dan *coefficient of determination* (R^2). Fungsi `mean_squared_error()` menghitung MSE berdasarkan nilai aktual `y_test` dan hasil prediksi `y_pred`, kemudian akar kuadratnya dihitung menggunakan `np.sqrt()` untuk memperoleh RMSE. Selanjutnya, fungsi `r2_score()` digunakan untuk menghitung nilai R^2. Berdasarkan hasil pengujian, diperoleh nilai RMSE sebesar 208296.73, yang menunjukkan rata-rata besarnya kesalahan prediksi model sekitar 208.297 satuan nilai target. Sementara itu, nilai R^2 sebesar 0.69946 menunjukkan bahwa model mampu menjelaskan sekitar 69.95% variasi pada variabel target, sedangkan sekitar 30.05% variasi lainnya belum dapat dijelaskan oleh model.

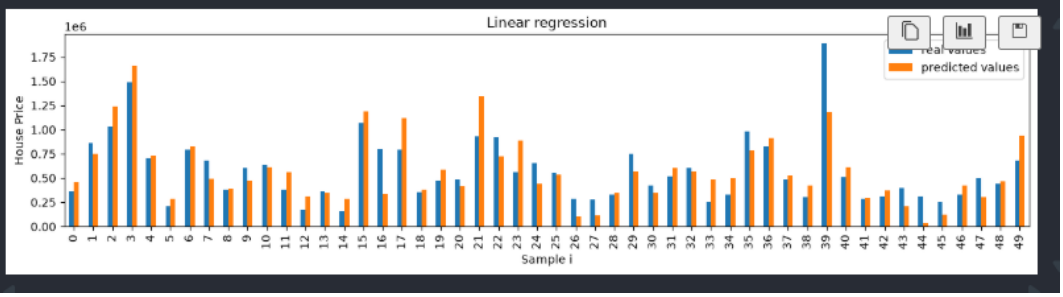
Tahap selanjutnya, dilakukan visualisasi untuk membandingkan antara nilai prediksi model dan nilai sebenarnya dari dataset testing.

```
data1 = pd.Series(y_test[:50].ravel())
data2 = pd.Series(y_pred[:50].ravel())

df_new = pd.concat([data1, data2], keys=['real values', 'predicted values'], axis=1)
#df_new.plot.bar() #bisa pakai cara 1
df_new.plot(kind='bar', figsize=(15,3)) # bisa pakai cara 2

plt.title("Linear regression")
plt.xlabel('Sample i')
plt.ylabel('House Price')
plt.show()
```

0.6s



Gambar 11: Visualisasi Prediksi vs Aktual Linear Regression

C.4.2 Decision Tree Regression

```
df_X = df.drop(['id', 'date','price'], axis=1)
df_y = df['price']

X = df_X.astype(float).values
y = df_y.astype(float).values


X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)
dt = DecisionTreeRegressor(max_depth=10)
dt.fit(X_train, y_train)

print('coef of determination training ',dt.score(X_train, y_train))
print('coef of determination testing ',dt.score(X_test, y_test))

print('prediction')
y_pred = dt.predict(X_test)
print(y_pred[ : 10])
print('real value')
print(y_test[0:10])
```

Gambar 12: Decision Tree Regression

Kode tersebut digunakan untuk membangun model *Decision Tree Regression* dalam memprediksi variabel `price`. Kolom `id`, `date`, dan `price` terlebih dahulu dihapus dari variabel prediktor, sedangkan `price` digunakan sebagai target. Data

kemudian dikonversi menjadi tipe `float` dan dibagi menjadi data latih sebesar 70% serta data uji sebesar 30%. Model `DecisionTreeRegressor` dibuat dengan parameter `max_depth=10`, yang membatasi kedalaman maksimum pohon hingga 10 tingkat, kemudian dilatih menggunakan `fit()`. Selanjutnya, `score()` digunakan untuk menghitung nilai R^2 pada data latih dan data uji. Terakhir, `predict()` digunakan untuk menghasilkan prediksi pada data uji, yang kemudian dibandingkan dengan 10 nilai aktual pertama dari `y_test`.

Outputnya dapat dilihat pada gambar 13.

```
coef of determination training  0.9185274272472612
coef of determination testing  0.7503727476221593
prediction
[ 372512.5         837693.06862745 1087247.96153846 1984617.64705882
  656314.08658009  237079.87179487  849106.21276596  517991.26363636
  366270.875       519039.40166667]
real value
[ 365000.  865000. 1038000. 1490000.  711000.  211000.  790000.  680000.
  384500.  605000.]
```

Gambar 13: Output Decision Tree Regression

Selanjutnya dilakukan evaluasi model menggunakan metrik RMSE dan R^2.

```
mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
r2 = r2_score(y_test, y_pred)

print('rmse :', rmse)
print('r2 :', r2)
```

✓ 0.0s

```
rmse : 189836.29952578805
r2 : 0.7503727476221593
```

Gambar 14: Evaluasi Model Decision Tree Regression

Kode tersebut digunakan untuk mengevaluasi performa model *Decision Tree Regression* menggunakan metrik RMSE dan R^2. Fungsi `mean_squared_error()`

menghitung nilai MSE antara nilai aktual `y_test` dan nilai prediksi `y_pred`, kemudian `np.sqrt()` digunakan untuk memperoleh nilai RMSE. Selanjutnya, fungsi `r2_score()` menghitung nilai *coefficient of determination* (R^2). Berdasarkan output, diperoleh nilai RMSE sebesar 189836.30, yang menunjukkan bahwa rata-rata besarnya kesalahan prediksi model sekitar 189.836 satuan nilai target. Nilai R^2 sebesar 0.75037 menunjukkan bahwa model mampu menjelaskan sekitar 75.04% variasi pada variabel target, sedangkan sekitar 24.96% variasi lainnya belum dapat dijelaskan oleh model. Dibandingkan hasil Linear Regression sebelumnya, model Decision Tree menghasilkan RMSE yang lebih rendah dan R^2 yang lebih tinggi, sehingga pada data pengujian tersebut Decision Tree memiliki performa prediksi yang lebih baik.

Tahap selanjutnya, dilakukan visualisasi untuk membandingkan antara nilai prediksi model dan nilai sebenarnya dari dataset testing.



Gambar 15: Visualisasi Prediksi vs Aktual Decision Tree Regression

C.4.3 Random Forest Regression

```
df_X = df.drop(['id', 'date','price'], axis=1)
df_y = df['price']

X = df_X.astype(float).values
y =df_y.astype(float).values

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)
rf = RandomForestRegressor()
rf.fit(X_train, y_train)

print('coef of determination training ', rf.score(X_train, y_train))
print('coef of determination testing ', rf.score(X_test, y_test))

print('prediction')
y_pred = rf.predict(X_test)
print(y_pred[:10])
print('real value')
print(y_test[0:10])
```

Gambar 16: Random Forest Regression

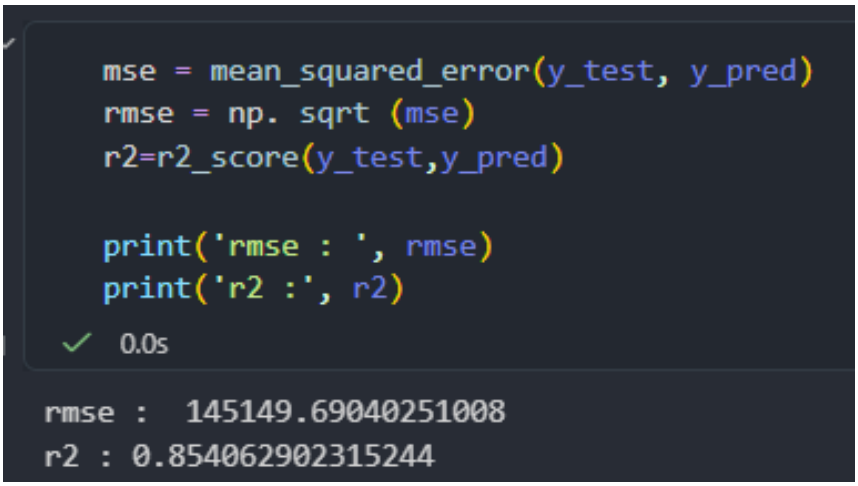
Kode tersebut digunakan untuk membangun model *Random Forest Regression* dalam memprediksi variabel `price`. Kolom `id`, `date`, dan `price` terlebih dahulu dihapus dari variabel prediktor, sedangkan `price` digunakan sebagai variabel target. Data kemudian dikonversi menjadi tipe `float` dan dibagi menjadi data latih sebesar 70% serta data uji sebesar 30% menggunakan `train_test_split()` dengan `random_state=42`. Model `RandomForestRegressor()` dibuat untuk membangun sejumlah pohon keputusan dan menggabungkan hasil prediksinya, kemudian dilatih menggunakan `fit()`. Selanjutnya, `score()` digunakan untuk menghitung nilai *coefficient of determination* (R^2) pada data latih dan data uji. Terakhir, metode `predict()` digunakan untuk menghasilkan prediksi harga berdasarkan data uji, yang kemudian dibandingkan dengan 10 nilai aktual pertama dari `y_test`.

Outputnya dapat dilihat pada gambar 17.

```
coef of determination training  0.9815968233189258
coef of determination testing  0.854062902315244
prediction
[ 386442.5   850296.11 1129688.4  2082598.    716246.16  241614.08
  858530.57  645387.86  397837.72  549111.49]
real value
[ 365000.  865000. 1038000. 1490000.  711000.  211000.  790000.  680000.
  384500.  605000.]
```

Gambar 17: Output Random Forest Regression

Selanjutnya dilakukan evaluasi model menggunakan metrik RMSE dan R^2.



Gambar 18: Evaluasi Model Random Forest Regression

Kode tersebut digunakan untuk mengevaluasi performa model *Random Forest Regression* menggunakan metrik RMSE dan R^2. Fungsi `mean_squared_error()` menghitung nilai MSE antara nilai aktual dan prediksi, kemudian `np.sqrt()` digunakan untuk memperoleh nilai RMSE. Selanjutnya, `r2_score()` menghitung nilai *coefficient of determination* (R^2). Berdasarkan output, diperoleh RMSE sebesar 145149.69, yang menunjukkan rata-rata kesalahan prediksi sekitar 145.150 satuan nilai target. Nilai R^2 sebesar 0.85406 menunjukkan bahwa model mampu menjelaskan sekitar 85.41% variasi pada variabel target, sedangkan sekitar 14.59% sisanya belum dapat dijelaskan oleh model. Hasil tersebut menunjukkan performa Random Forest yang lebih baik dibandingkan Linear Regression dan Decision Tree berdasarkan nilai

RMSE yang lebih rendah dan R^2 yang lebih tinggi.

Selanjutnya, dilakukan visualisasi untuk membandingkan antara nilai prediksi model dan nilai sebenarnya dari dataset testing.



Gambar 19: Visualisasi Prediksi vs Aktual Random Forest Regression

Visualisasi tersebut membandingkan nilai aktual (*real values*) dengan nilai hasil prediksi (*predicted values*) dari model *Random Forest Regression* pada 50 sampel data uji. Sumbu-x menunjukkan nomor sampel, sedangkan sumbu-y menunjukkan nilai *House Price*. Batang yang berdekatan menunjukkan perbandingan antara harga aktual dan harga yang diprediksi oleh model. Secara umum, sebagian besar batang prediksi memiliki nilai yang relatif dekat dengan nilai aktual, meskipun terdapat beberapa perbedaan yang cukup besar, seperti pada sampel ke-3. Kedekatan antara kedua batang menunjukkan bahwa model mampu menghasilkan prediksi yang cukup mendekati nilai sebenarnya, yang sejalan dengan hasil evaluasi sebelumnya berupa nilai RMSE sebesar 145149.69 dan R^2 sebesar 0.85406.

D Tugas dan Analisis

D.1 Tujuan Penggunaan Dataset

Diberikan dataset dengan link berikut `https://www.kaggle.com/datasets/hellbuoy/car-price-prediction/data`.

```
import pandas as pd

df = pd.read_csv('car-price/CarPrice_Assignment.csv')
```

Python

```
df.head()
```

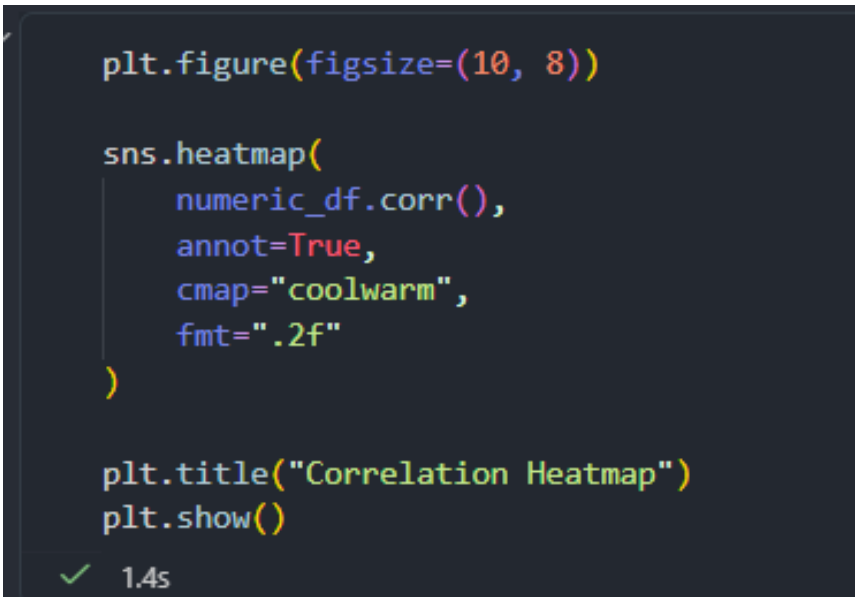
Python

car_ID	symboling	CarName	fueltype	aspiration	doornumber	carbody	drivewheel	enginelocation	wheelbase	...	enginesize	fuelsystem	boreratio	stroke	compressionratio	horsepower	peakrpm	citympg	highwaympg	price
1	3	alfa-romero giulia	gas	std	two	convertible	rwd	front	88.6	...	130	mpfi	3.47	2.68	9.0	111	5000	21	27	13495.0
2	3	alfa-romero stelvio	gas	std	two	convertible	rwd	front	88.6	...	130	mpfi	3.47	2.68	9.0	111	5000	21	27	16500.0
3	1	alfa-romero Quadrifoglio	gas	std	two	hatchback	rwd	front	94.5	...	152	mpfi	2.68	3.47	9.0	154	5000	19	26	16500.0
4	2	audi 100 ls	gas	std	four	sedan	fwd	front	99.8	...	109	mpfi	3.19	3.40	10.0	102	5500	24	30	13950.0
5	2	audi 100ls	gas	std	four	sedan	4wd	front	99.4	...	136	mpfi	3.19	3.40	8.0	115	5500	18	22	17450.0

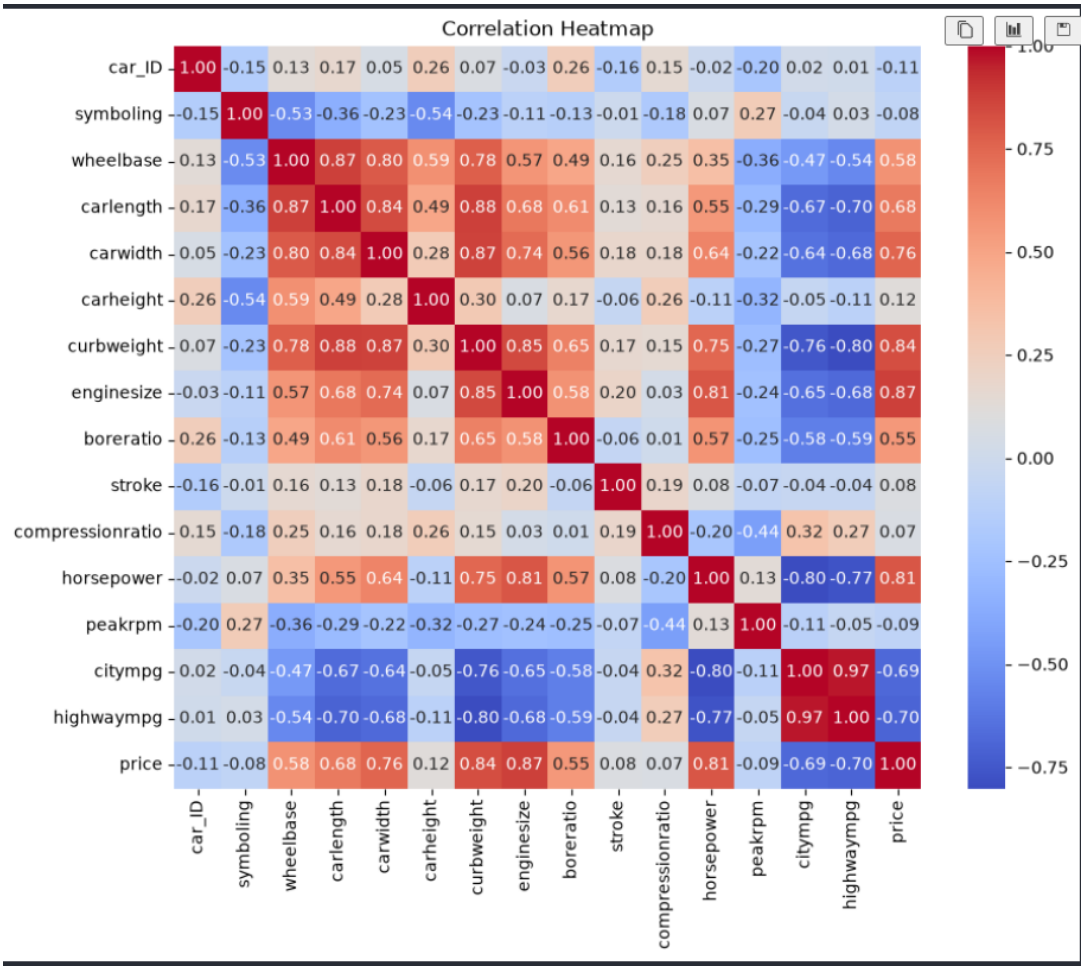
Gambar 20: Dataset Car Price Prediction

Dataset Car Price Prediction digunakan untuk menganalisis faktor-faktor yang memengaruhi harga sebuah mobil serta membangun model machine learning yang dapat memprediksi harga mobil berdasarkan karakteristiknya. Dataset terdiri dari berbagai atribut kendaraan seperti ukuran kendaraan, berat kendaraan, jenis mesin, kapasitas mesin, horsepower, konsumsi bahan bakar, dan atribut kategorikal lainnya. Target atau output yang digunakan adalah atribut price, sedangkan atribut lainnya digunakan sebagai input atau variabel independen. car_ID tidak digunakan karena hanya merupakan identitas unik data, sedangkan CarName tidak digunakan dalam eksperimen ini karena memiliki banyak kategori unik.

D.2 Variabel yang paling mempengaruhi output



Gambar 21: Korelasi Atribut Numerik Dataset Car Price Prediction



Gambar 22: Heatmap Korelasi Atribut Numerik Dataset Car Price Prediction

```
correlation.drop("price").abs().sort_values(ascending=False).head(5)
✓ 0.0s

enginesize     0.874145
curbweight     0.835305
horsepower     0.808139
carwidth       0.759325
highwaympg     0.697599
Name: price, dtype: float64
```

Gambar 23: Visualisasi Korelasi Atribut Numerik Dataset Car Price Prediction

Dapat dilihat pada heatmap korelasi pada gambar 23 bahwa Output tersebut menunjukkan lima variabel yang memiliki nilai korelasi absolut tertinggi terhadap variabel `price`. Variabel `enginesize` memiliki korelasi sebesar 0.874145, diikuti

oleh `curbweight` sebesar 0.835305, `horsepower` sebesar 0.808139, `carwidth` sebesar 0.759325, dan `highwaympg` sebesar 0.697599. Karena seluruh nilai korelasi bernilai positif, kelima variabel tersebut memiliki hubungan positif dengan harga mobil. Variabel `enginesize` memiliki korelasi paling tinggi, sehingga dapat dikatakan sebagai variabel yang memiliki hubungan paling kuat dengan `price` dibandingkan variabel lainnya pada hasil tersebut.

D.3 Model Regresi

D.3.1 Linear Regression

```
X_train_processed = preprocessor.fit_transform(X_train)
X_test_processed = preprocessor.transform(X_test)

linear_model = LinearRegression()

linear_model.fit(
    X_train_processed,
    y_train
)

y_pred_lr = linear_model.predict(X_test_processed)
```

✓ 0.1s

```
rmse_lr = np.sqrt(mean_squared_error(y_test, y_pred_lr))
r2_lr = r2_score(y_test, y_pred_lr)

r_lr = np.corrcoef(y_test, y_pred_lr)[0, 1]

print("Linear Regression")
print("RMSE :", rmse_lr)
print("R2   :", r2_lr)
print("R    :", r_lr)
```

✓ 0.0s

```
Linear Regression
RMSE : 2798.994302875281
R2   : 0.8869244515927133
R    : 0.9433408832996859
```

Gambar 24: Linear Regression

D.3.2 Decision Tree Regression

```
tree_model = DecisionTreeRegressor(
    random_state=42
)

tree_model.fit(
    X_train_processed,
    y_train
)

y_pred_tree = tree_model.predict(X_test_processed)
```

✓ 0.0s

```
rmse_tree = np.sqrt(
    mean_squared_error(y_test, y_pred_tree)
)

r2_tree = r2_score(
    y_test,
    y_pred_tree
)

r_tree = np.corrcoef(
    y_test,
    y_pred_tree
)[0, 1]

print("Decision Tree Regression")
print("RMSE :", rmse_tree)
print("R2   :", r2_tree)
print("R    :", r_tree)
```

✓ 0.0s

```
Decision Tree Regression
RMSE : 2888.7645518509235
R2   : 0.8795549472580222
R    : 0.9499542864252319
```

Gambar 25: Decision Tree Regression

D.3.3 Random Forest Regression

```
rf_model = RandomForestRegressor(
    n_estimators=100,
    random_state=42
)

rf_model.fit(
    X_train_processed,
    y_train
)

y_pred_rf = rf_model.predict(X_test_processed)
```

✓ 0.3s

```
rmse_rf = np.sqrt(
    mean_squared_error(y_test, y_pred_rf)
)

r2_rf = r2_score(
    y_test,
    y_pred_rf
)

r_rf = np.corrcoef(
    y_test,
    y_pred_rf
)[0, 1]

print("Random Forest Regression")
print("RMSE :", rmse_rf)
print("R2   :", r2_rf)
print("R    :", r_rf)
```

✓ 0.0s

```
Random Forest Regression
RMSE : 1975.343398573095
R2   : 0.9436816549343897
R    : 0.9722340170766439
```


Gambar 26: Random Forest Regression

D.4 Perbandingan performa Model

```
results = pd.DataFrame({
    "model": [
        "Linear Regression",
        "Decision Tree Regression",
        "Random Forest Regression"
    ],

    "RMSE": [
        rmse_lr,
        rmse_tree,
        rmse_rf
    ],

    "R2": [
        r2_lr,
        r2_tree,
        r2_rf
    ],

    "R": [
        r_lr,
        r_tree,
        r_rf
    ]
})

results
```

✓ 0.0s

	model	RMSE	R2	R
0	Linear Regression	2798.994303	0.886924	0.943341
1	Decision Tree Regression	2888.764552	0.879555	0.949954
2	Random Forest Regression	1975.343399	0.943682	0.972234

Gambar 27: Perbandingan Performa Model

Output tersebut menunjukkan perbandingan performa tiga model regresi berdasarkan metrik RMSE, R^2, dan koefisien korelasi (R). *Random Forest Regression* memiliki performa terbaik dengan RMSE sebesar 1975.34, R^2 sebesar 0.943682, dan R sebesar 0.972234. Sementara itu, *Linear Regression* memperoleh RMSE 2798.99 dan

R^2 0.886924, sedangkan *Decision Tree Regression* memperoleh RMSE 2888.76 dan R^2 0.879555. Dengan demikian, *Random Forest Regression* merupakan model terbaik karena memiliki nilai RMSE paling rendah serta nilai R^2 dan R paling tinggi dibandingkan kedua model lainnya.

D.5 Hasil Eksperimen

Berdasarkan hasil eksperimen, Random Forest Regression merupakan model dengan performa terbaik dibandingkan Linear Regression dan Decision Tree Regression. Hal tersebut ditunjukkan oleh nilai RMSE yang paling rendah serta nilai R^2 dan R yang paling tinggi. RMSE yang rendah menunjukkan bahwa kesalahan prediksi harga relatif kecil, sedangkan nilai R^2 yang tinggi menunjukkan bahwa model mampu menjelaskan variasi harga dengan lebih baik. Random Forest juga terdiri dari banyak Decision Tree sehingga mampu menangkap hubungan non-linear antara karakteristik kendaraan dan harga dengan lebih baik dibandingkan Linear Regression yang hanya memodelkan hubungan linear dan satu Decision Tree yang lebih rentan terhadap overfitting.

E Kesimpulan

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa model Random Forest Regression memiliki performa terbaik dibandingkan Linear Regression dan Decision Tree Regression. Hal ini ditunjukkan oleh nilai RMSE yang lebih rendah dan nilai R^2 yang lebih tinggi pada data uji. Model Random Forest mampu menangkap hubungan non-linear dan interaksi antarvariabel dengan lebih baik, sehingga menghasilkan prediksi yang lebih akurat terhadap harga rumah.

Berdasarkan hasil eksperimen, pemilihan model regresi sebaiknya mempertimbangkan karakteristik data, kompleksitas hubungan antarvariabel, serta kebutuhan interpretasi model. Random Forest Regression dapat menjadi pilihan yang baik ketika performa prediksi menjadi prioritas utama, meskipun memiliki kompleksitas yang lebih tinggi dibandingkan Linear Regression dan Decision Tree Regression.

F Daftar Pustaka

[1] G. James, D. Witten, T. Hastie, and R. Tibshirani, *An Introduction to Statistical Learning: with Applications in R*, 2nd ed. New York: Springer, 2021. [Online]. Available: https://www.statlearning.com/

[2] Scikit-learn Developers, "Linear Regression — Scikit-learn documentation," https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LinearRegression.html, 2026, [Accessed 11-09-2026].

[3] ——, "Decision Tree Regressor — Scikit-learn documentation," https://scikit-learn.org/stable/modules/generated/sklearn.tree.DecisionTreeRegressor.html, 2026, [Accessed 11-09-2026].

[4] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001. [Online]. Available: https://doi.org/10.1023/A:1010933404324

[5] Scikit-learn Developers, "Metrics and scoring: Quantifying the quality of predictions — Scikit-learn documentation," https://scikit-learn.org/stable/api/sklearn.metrics.html, 2026, [Accessed 11-09-2026].

[6] National Institute of Standards and Technology, "Motivation: How do we construct a goodness-of-fit metric for a model?" https://www.itl.nist.gov/div898/handbook/pri/section5/pri5992.htm, 2026, [Accessed 11-09-2026].